

1. Why is it better to code against an interface rather than a concrete class?

It makes the code more flexible and easier to change. Different classes can implement the same interface without changing the code that uses them.

2. When should you prefer an abstract class over an interface?

Use an abstract class when related classes share common data or implementation, and you want to provide some common behavior.

3. How does implementing `Comparable` improve flexibility in sorting?

It defines a default way to compare objects. This allows arrays or collections of the objects to be sorted directly using their comparison rule.

4. What is the primary purpose of a copy constructor in C#?

Its main purpose is to create a new object based on the values of an existing object.

5. How does explicit interface implementation help in resolving naming conflicts?

It allows a class to have different implementations for methods with the same name from different interfaces, avoiding naming conflicts.

6. What is the key difference between encapsulation in structs and classes?

The main difference is that structs are value types, while classes are reference types. Both can use private fields and public properties to achieve encapsulation.

7. What is abstraction as a guideline, and what's its relation with encapsulation?

Abstraction means showing only the important details and hiding unnecessary implementation details. Encapsulation protects the internal data and controls how it is accessed. They work together to make code easier to use and maintain.

8. How do default interface implementations affect backward compatibility in C#?

They allow new methods to be added to an interface without forcing all existing implementing classes to implement those methods immediately.

9. How does constructor overloading improve class usability?

It gives users different ways to create an object depending on the data they have, making the class easier and more flexible to use.